Community Infrastructure Reporting Platform (CIRP)

1. INTRODUCTION

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to define the functional and non-functional requirements of the Community Infrastructure Reporting Platform (CIRP).

CIRP is a digital platform designed to help citizens report infrastructure problems in their communities and enable municipal authorities to manage, assign, monitor, and resolve reported issues.

The system consists of three main components:

1. A mobile application for citizens.
2. A web-based administrative platform for municipal authorities.
3. A backend API and AI service for managing reports and identifying potential duplicate reports.

This document describes the system's objectives, users, functional requirements, system constraints, interfaces, security requirements, and future improvements.

1.2 Scope

The Community Infrastructure Reporting Platform provides a centralized system for reporting and managing community infrastructure problems.

Citizens can submit reports containing descriptions, categories, geographical locations, and photographs. Municipal administrators can review reports, assign tasks, set priorities, and monitor the progress of maintenance activities.

The system supports infrastructure categories such as:

- Road damage
- Water leakage
- Drainage problems
- Garbage and waste management
- Streetlight failure
- Public facility damage
- Other infrastructure problems

The system also includes an AI-assisted duplicate detection service that compares new reports with existing reports using text similarity, category matching, and geographical distance.

The main goal is to improve communication between citizens and municipal authorities, reduce duplicate reports, and support more efficient infrastructure maintenance.

1.3 Objectives

The main objectives of CIRP are:

1. To provide citizens with a convenient way to report infrastructure problems.
2. To collect report descriptions, photographs, and geographical locations.
3. To help municipal authorities manage and prioritize reports.
4. To support task assignment to responsible staff or contractors.
5. To detect potentially duplicated reports.
6. To track report and maintenance-task progress.
7. To improve accountability and transparency in infrastructure management.
8. To maintain records of reports, users, tasks, and related activities.

2. OVERALL SYSTEM DESCRIPTION

2.1 Product Perspective

CIRP is a client-server application composed of a mobile application, an administrative web application, a backend API, a database, and an AI service.

The mobile application allows citizens to submit reports and monitor their status.

The web application allows municipal administrators to review reports, manage infrastructure issues, create tasks, and oversee department activities.

The backend handles authentication, authorization, report management, database operations, and communication between the applications and the AI service.

The AI service analyzes report descriptions and compares them with existing reports to identify possible duplicates.

2.2 System Components

2.2.1 Citizen Mobile Application

The mobile application is developed using Flutter and Dart.

Its main responsibilities are:

- User registration and login.
- Email and password authentication.
- OTP verification where supported by the implemented authentication flow.
- Password reset.
- Submission of infrastructure reports.
- Collection of GPS coordinates and location information.
- Selection of infrastructure categories.
- Uploading photographs from the device gallery.
- Viewing previously submitted reports.
- Tracking report status.
- Viewing report details and associated photographs.
- Checking for possible duplicate reports.

2.2.2 Municipal Administrative Web Application

The administrative web application is developed using Next.js, React, TypeScript, Tailwind CSS, and shadcn/ui.

Its main responsibilities are:

- Displaying dashboard statistics.
- Viewing submitted reports.
- Searching and filtering reports.
- Viewing report details and photographs.
- Accepting or rejecting reports.
- Providing reasons for rejection or disputes.
- Creating maintenance tasks.
- Assigning tasks to workers or contractors.
- Setting task priorities and deadlines.
- Recording work notes and progress.
- Linking reports to infrastructure assets.
- Managing users, staff, and contractors.
- Supporting department-specific report management.

2.2.3 Backend API

The backend is developed using Node.js and Express.js.

Its main responsibilities are:

- User authentication and authorization.
- Report creation, retrieval, updating, and management.
- User and role management.
- Report searching, filtering, and pagination.
- Image data storage and retrieval.
- Department routing based on report categories.
- Duplicate detection requests.
- Maintenance task creation and assignment.
- Infrastructure asset management.
- Notification-related data management.
- Recording relevant system activities.

2.2.4 AI Duplicate Detection Service

The AI service is developed using Python, FastAPI, and scikit-learn.

Its responsibilities are:

- Receiving report information for duplicate checking.
- Comparing report descriptions using TF-IDF and cosine similarity.
- Comparing report categories.
- Considering geographical distance.
- Calculating similarity scores.
- Returning potential duplicate reports.
- Providing the highest-ranking matches.

The current implementation focuses on text, category, and location-based comparisons. It does not yet perform image-based duplicate detection.

2.3 User Classes and Characteristics

2.3.1 Citizen

Citizens use the mobile application to report infrastructure problems and monitor their reports.

Their permissions include:

- Registering and logging in.
- Creating infrastructure reports.

- Viewing their own reports.
- Viewing report details and statuses.
- Accessing available report information.

Citizens must not access administrative functions or other users' private information.

2.3.2 Department Administrator

Department administrators manage reports assigned to their authorized departments.

Their responsibilities include:

- Reviewing assigned reports.
- Accepting or rejecting reports.
- Providing rejection reasons.
- Creating maintenance tasks.
- Assigning work to staff or contractors.
- Setting priorities and deadlines.
- Monitoring maintenance progress.
- Viewing department-related information.

Department administrators must only access information permitted by their assigned roles and departmental responsibilities.

2.3.3 Super Administrator

The super administrator manages the overall platform.

Responsibilities include:

- Managing users and roles.
- Managing departments and system-wide information.
- Monitoring reports across departments.
- Managing administrative access.
- Overseeing system operations.

2.3.4 Field Worker

Field workers are expected to perform assigned maintenance activities and report their progress.

A complete, separate field-worker workflow is not currently implemented. Worker-related activities are presently managed through the administrative functions.

3. FUNCTIONAL REQUIREMENTS

Functional requirements describe what the system must do.

3.1 User Registration and Authentication

FR-01: User Registration

The system shall allow citizens to create accounts using the registration information supported by the application.

FR-02: User Login

The system shall authenticate users using valid login credentials.

FR-03: Password Reset

The system shall provide a password-reset process.

FR-04: Authentication Tokens

The backend shall issue JWT authentication tokens to authenticated users and validate them when protected resources are accessed.

FR-05: Role-Based Access Control

The system shall restrict functions and data according to user roles and permissions.

3.2 Infrastructure Report Management

FR-06: Submit Report

The system shall allow citizens to submit infrastructure reports containing a title, description, category, location information, and an optional photograph, according to the supported submission form.

FR-07: Geographic Location

The system shall support the collection of GPS coordinates and related location information for reports.

FR-08: Photograph Upload

The mobile application shall allow citizens to select a photograph from their device gallery and attach it to a report.

FR-09: View My Reports

The system shall allow citizens to view reports they have submitted.

FR-10: Report Details

The system shall display available details about a report, including its description, category, location, photograph, and status.

FR-11: Search and Filter Reports

The administrative platform shall support searching and filtering reports using available report attributes.

FR-12: Report Review

Authorized administrators shall be able to review submitted reports and accept or reject them.

FR-13: Rejection Reason

The system shall allow authorized administrators to provide a reason when rejecting or disputing a report.

3.3 Duplicate Detection

FR-14: Duplicate Report Checking

The system shall check a new report against selected existing reports to identify potential duplicates.

FR-15: Text Similarity

The AI service shall use TF-IDF and cosine similarity to compare report descriptions.

FR-16: Category and Location Comparison

The duplicate detection service shall consider category matching and geographical distance when identifying potential duplicates.

FR-17: Duplicate Response

The service shall return similarity information and potential matching reports. The current implementation uses a configurable comparison process with a documented 75% similarity threshold and a location condition of less than 500 meters.

FR-18: Duplicate Submission Handling

When the backend identifies a duplicate according to its current rules, it shall reject the new submission and return an HTTP 409 response.

3.4 Task and Maintenance Management

FR-19: Create Maintenance Tasks

Authorized administrators shall be able to create tasks associated with infrastructure reports.

FR-20: Assign Tasks

The system shall support assigning tasks to available workers or contractors through the administrative platform.

FR-21: Task Priority and Deadline

The system shall allow authorized administrators to set task priorities and deadlines.

FR-22: Task Progress

The system shall support recording and viewing available task progress information and work notes.

FR-23: Infrastructure Asset Linking

The system shall support linking reports to relevant infrastructure assets where applicable.

3.5 Administrative Management

FR-24: Dashboard

The administrative platform shall display available summary statistics about reports and maintenance activities.

FR-25: Department Management

The system shall support department-related report routing and administrative views.

FR-26: User and Staff Management

The administrative platform shall provide available functions for viewing and managing users, staff, and contractors.

FR-27: Activity Records

The system shall maintain relevant audit records for supported system activities.

Note: The requirements above describe the intended system behavior. Some functions are implemented, some are partially implemented, and others require further development and testing.

4. BUSINESS RULES

4.1 Report Submission Rules

1. A citizen must be authenticated before submitting a report through protected report-submission functions.
2. A report must contain the required information specified by the submission form.
3. The report category must be selected from the supported categories.
4. Location information should be provided according to the application's validation rules.
5. Photographs must follow the file formats and size limits supported by the application.
6. Citizens must not be allowed to modify reports belonging to other citizens unless an authorized administrative function permits the action.

4.2 Duplicate Detection Rules

The current duplicate detection process uses the following criteria:

1. Report description similarity.
2. Report category matching.
3. Geographic distance between reports.
4. A similarity threshold of 75%.
5. A location condition of less than 500 meters.
6. Comparison against a limited set of recent reports, currently documented as up to 50 reports.

The system uses these criteria to identify potential duplicates. Because the current approach is rule-based and text-focused, it may incorrectly classify reports as duplicates or fail to identify some actual duplicates.

4.3 Department Routing Rules

Reports should be routed to the department responsible for the selected infrastructure category.

Examples include:

- Road damage → Responsible road maintenance department.
- Water leakage → Responsible water service department.
- Streetlight failure → Responsible electricity or streetlight maintenance department.
- Garbage and waste → Responsible waste management department.
- Drainage problems → Responsible drainage or public works department.

The final routing must follow the actual municipal organizational structure and configured department responsibilities.

4.4 Task Management Rules

1. Maintenance tasks must be associated with relevant reports when required.
2. Only authorized administrators may create or assign tasks through the currently implemented administrative functions.
3. Tasks should include priorities and deadlines when required.
4. Task progress should be recorded to support maintenance monitoring.
5. The platform should preserve relevant records of maintenance activities.

5. SYSTEM WORKFLOW

5.1 Citizen Reporting Workflow

1. The citizen opens the mobile application.
2. The citizen registers or logs in.
3. The citizen selects the option to submit a report.
4. The citizen enters the report title and description.
5. The citizen selects the infrastructure category.
6. The citizen provides location information.
7. The citizen selects a photograph from the gallery if needed.
8. The system sends the report information to the backend.
9. The backend checks the report for potential duplicates.
10. If the report is accepted by the submission rules, the backend stores it.
11. The report is routed according to the configured category and department rules.
12. The citizen can view the submitted report and its available status information.

5.2 Administrative Workflow

1. An administrator logs into the administrative platform.
2. The administrator views the dashboard.
3. The administrator searches for or selects a report.
4. The administrator reviews its details, location, and photograph.

5. The administrator accepts or rejects the report.
6. If rejected, the administrator provides a reason where required.
7. If maintenance is needed, the administrator creates a task.
8. The administrator assigns the task to an appropriate worker or contractor.
9. The administrator sets the priority and deadline.
10. The administrator records or reviews available progress information.
11. The report and task records are updated as work progresses.

6. EXTERNAL INTERFACE REQUIREMENTS

6.1 Mobile Application Interface

The mobile application shall provide interfaces for:

- Registration and login.
- Password reset.
- Report creation.
- Category selection.
- GPS and location information.
- Photograph selection.
- Report history.
- Report details and status.
- Map-based location viewing.

The mobile application is developed using Flutter and Dart for Android. iOS compatibility has not yet been verified.

6.2 Administrative Web Interface

The web application shall provide interfaces for:

- Authentication.
- Dashboard statistics.
- Report lists and filters.
- Report details.
- Report review and rejection.
- Maintenance task management.
- Worker and contractor assignment.
- Department-related views.
- User and staff management.

The interface is developed using Next.js, React, TypeScript, Tailwind CSS, and shadcn/ui.

6.3 Backend Interface

The backend shall expose API endpoints for the mobile application and administrative web application.

The API shall support authentication, report management, task management, and other implemented functions.

The backend is developed using Node.js and Express.js.

6.4 AI Service Interface

The AI service shall expose HTTP endpoints for duplicate detection and health checking.

Current documented endpoints include:

- `POST /ai/detect-duplicate/`
- `POST /ai/detect-duplicate-single/`
- `GET /ai/health/`
- `/docs` for interactive API documentation.

The AI service is developed using Python, FastAPI, and scikit-learn.

6.5 Database Interface

The current backend uses SQLite to store application data.

The database contains entities related to:

- Users.
- Reports.
- Report photographs.
- Categories.
- Departments.
- Maintenance tasks.
- Infrastructure assets.
- Notifications.
- Audit logs.
- Staff and contractors.

A production deployment should evaluate migration to PostgreSQL or another suitable production database.

7. DATA REQUIREMENTS

7.1 User Data

The system stores user-related information required for authentication, authorization, and account management.

Passwords must be stored as secure password hashes rather than plain text.

7.2 Report Data

A report may contain:

- Report identifier.
- Citizen identifier.
- Title.
- Description.
- Category.
- Latitude and longitude.
- Address or landmark information.
- Photograph data or references.
- Report status.
- Creation and update timestamps.
- Department association, where applicable.

7.3 Maintenance Task Data

A task may contain:

- Task identifier.
- Related report identifier.
- Assigned worker or contractor.
- Task description.
- Priority.
- Deadline.
- Status or progress information.
- Work notes.
- Creation and update timestamps.

7.4 Infrastructure Asset Data

Infrastructure asset records support the association of reports with relevant infrastructure items.

The exact asset fields shall follow the implemented database schema and future municipal requirements.

7.5 Data Quality

The system should validate required fields, reject invalid input, protect data integrity, and prevent unauthorized modification of records.

The system should also maintain accurate relationships between reports, users, departments, assets, and maintenance tasks.

8. NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements describe the quality, security, and operational expectations of the system.

8.1 Security Requirements

NFR-01: The system shall authenticate users before allowing access to protected functions.

NFR-02: The system shall enforce role-based access control.

NFR-03: Passwords shall be stored using secure hashing.

NFR-04: Authentication tokens shall expire according to the configured expiration policy.

NFR-05: The backend shall validate incoming requests and reject invalid or unauthorized operations.

NFR-06: The system should use HTTPS in deployed environments.

NFR-07: The system should implement rate limiting to reduce abuse of authentication and public API endpoints.

NFR-08: The system should validate and sanitize user input to reduce common web application vulnerabilities.

NFR-09: The system should protect private user information and prevent unauthorized access to reports.

NFR-10: The system should maintain audit records for important administrative actions.

Current status: Authentication, JWT, bcrypt password hashing, and role-based access controls are implemented. HTTPS, rate limiting, comprehensive input sanitization, and other production security measures require additional implementation and testing.

8.2 Performance Requirements

NFR-11: The system should provide acceptable response times for report submission, retrieval, and administrative operations under the expected workload.

NFR-12: The system should efficiently retrieve reports using pagination, filtering, and search.

NFR-13: The duplicate detection service should process report comparisons within an acceptable time.

NFR-14: The system should be evaluated under realistic workloads before production deployment.

The current implementation has development-level performance measurements. Production performance targets must be established through testing.

8.3 Reliability Requirements

NFR-15: The system should preserve stored reports and related records during normal operation.

NFR-16: The system should handle invalid requests and service errors without crashing unnecessarily.

NFR-17: The production system should include regular database backups and a tested recovery process.

NFR-18: The system should provide useful error messages when the backend or AI service is unavailable.

8.4 Usability Requirements

NFR-19: The mobile application should provide a simple reporting process for citizens.

NFR-20: The administrative platform should organize reports and tasks clearly.

NFR-21: Forms should provide validation messages for missing or invalid information.

NFR-22: The interface should be usable on common supported devices and screen sizes.

NFR-23: The platform should support the languages required by its intended users.

The broader project intends to support English, Amharic, Afaan Oromo, and Tigrinya in the citizen application. AI duplicate detection is currently English-focused.

8.5 Maintainability

NFR-24: The system should use modular components to simplify maintenance.

NFR-25: The backend, mobile application, web application, and AI service should have documented interfaces.

NFR-26: The project should include setup instructions and appropriate testing documentation.

8.6 Scalability

NFR-27: The system should be designed to support increasing numbers of users and reports.

NFR-28: The production architecture should support an appropriate database and image-storage solution.

NFR-29: The system should be evaluated for increased traffic and storage requirements before expanding to additional municipalities.

8.7 Compatibility

NFR-30: The mobile application shall be tested on supported Android devices.

NFR-31: The administrative web application should support modern web browsers.

NFR-32: iOS compatibility shall be verified separately if iOS deployment is required.

9. SYSTEM CONSTRAINTS AND LIMITATIONS

9.1 Mobile Application Limitations

- Direct camera capture is currently disabled because of memory limitations encountered during development.
- Citizens currently select photographs from the device gallery.
- The implemented report flow supports one photograph per report.
- Android has been tested, while iOS compatibility remains unverified.
- Local backend connectivity may require ADB reverse or the computer's local IP address.

9.2 Backend Limitations

- SQLite is currently used instead of a production-oriented database deployment.
- Photographs are stored as base64 data in the database.
- Email delivery is incomplete or not implemented.
- Push notifications are not implemented.
- The backend currently runs as a single development server.
- HTTPS is not configured in the local development environment.

9.3 Administrative Web Limitations

- The notifications page has a schema mismatch.
- Analytics and reporting capabilities are limited.
- Bulk operations and CSV/Excel exports are not fully implemented.
- Mobile responsiveness requires additional work.
- Real-time updates using WebSockets are not implemented; some information requires manual refresh.

9.4 AI Limitations

- Duplicate detection uses TF-IDF and cosine similarity rather than a trained semantic machine-learning model.
- Image similarity is not implemented.
- Automatic image category classification is not integrated into the current duplicate detection service.
- The current text-based approach is English-focused.
- The service uses CPU processing.
- The similarity threshold and distance rules may require adjustment based on real-world data.

9.5 Deployment Limitations

The system is currently intended for local development and controlled testing.

A production deployment requires additional work, including:

- Production database configuration.
- HTTPS and domain configuration.
- Secure image storage.
- Backup and recovery.
- Monitoring and logging.
- Deployment automation.
- Security testing.
- Performance and scalability testing.

10. ACCEPTANCE CRITERIA

The system may be considered ready for a controlled pilot when the following criteria have been verified.

1. Citizens can register and authenticate successfully.
2. Authenticated citizens can submit valid infrastructure reports.
3. Report information and location data are stored correctly.
4. Citizens can view their submitted reports and available statuses.
5. Authorized administrators can access the administrative platform.
6. Administrators can review reports and accept or reject them.
7. Rejection reasons can be recorded where required.
8. Administrators can create maintenance tasks and assign them to workers or contractors through supported functions.
9. Task priorities, deadlines, and progress information can be managed.
10. The duplicate detection service can compare reports and return potential matches.
11. The backend handles duplicate submissions according to the configured rules.
12. Unauthorized users cannot access protected administrative functions.
13. The system handles invalid input and expected errors appropriately.
14. The mobile application and administrative web application can communicate with the backend in the target environment.
15. The system has been tested using representative use cases.

Important: Passing these criteria requires actual testing and documented results. The existence of a feature in the code alone does not prove that it meets the acceptance criteria.

11. FUTURE ENHANCEMENTS

The following enhancements are recommended for future development.

11.1 Mobile Application

- Enable direct camera capture where supported devices can handle it.
- Support multiple photographs per report.
- Improve offline reporting and synchronization.
- Add push notifications.
- Complete multilingual support and test each language.
- Improve location selection and GPS error handling.

11.2 Administrative Platform

- Implement real-time updates.
- Improve mobile responsiveness.
- Add advanced analytics and reporting.
- Implement CSV and Excel exports.
- Improve notification management.
- Develop a dedicated field-worker workflow.
- Add detailed department performance reports.

11.3 AI and Machine Learning

- Improve duplicate detection using semantic text embeddings.
- Add image similarity for infrastructure photographs.
- Train and evaluate an image classification model for infrastructure categories.
- Expand multilingual text processing.
- Create a labeled dataset for evaluating duplicate detection.
- Measure precision, recall, F1-score, and false-positive rates.
- Tune thresholds using real-world municipal reports.

11.4 Security and Deployment

- Migrate to PostgreSQL for production.
- Configure HTTPS.
- Implement rate limiting and stronger request validation.
- Complete security testing.
- Add secure image/object storage.
- Implement automated backups and recovery.
- Add centralized logging and monitoring.
- Establish CI/CD deployment workflows.
- Conduct load and performance testing.

12. CONCLUSION

The Community Infrastructure Reporting Platform is intended to improve the process of identifying, reporting, managing, and resolving infrastructure problems in communities.

The platform combines a citizen mobile application, a municipal administrative web application, a backend API, a database, and an AI-assisted duplicate detection service.

The current implementation provides core report submission, report management, administrative review, task management, and text-based duplicate detection capabilities. However, several areas require additional development, testing, and security improvements before the platform can be used in a full production environment.

This SRS serves as a foundation for continued implementation, testing, evaluation, and future development of CIRP.

Prepared by: CIRP Development Team

Rediet Sharew……CTC 573-26
Rediet Asfer……...CTC 3755-26
Meron……..CTC 380-26
Meron……CTC354-26